Análisis de Algoritmos 2026/2027

# Práctica 1

Jarque Narro, Javier. González Hijano, Iván.

Grupo: 126

| Código | Gráficas | Memoria | Total |
|---|---|---|---|
| | | | |

# 1. Introducción

En esta práctica hemos programado en c funciones para generar permutaciones (permutaciones.c), hemos implementado las funciones InsertSort y BubbleSort y visto su número de operaciones básicas (sorting.c) y hemos medido tiempos(times.c). Así hemos comparado ambos algoritmos y visto sus diferencias.

Programas de prueba, entradas y salidas:

- exercise1: -LimInf -limSup - numN. Imprime numN números aleatorios entre liminf y limSup.
- exercise2 y exercise3: - size - numP. Imprimen numP permutaciones de size elementos (exercise2
  con generate_perm y exercise3 con generate_permutations).
- exercise4 y 4b: - size. Genera una permutación, la ordena e imprime el resultado (debe salir 1 2 ... N).
- exercise5 y 5b: - num_min -num_max -incr -numP - outputFile. Guarda en un fichero una fila por tamaño con seis columnas: N, n_elems, tiempo medio de CPU (s), OB media, OB mínima y OB máxima.

# 2. Herramientas y metodología

Los entornos que hemos utilizado son Windows, Linux, MacOS y las herramientas principales usadas han sido Visual Studio, Code, gcc, Valgrind, Gnuplot y Git).

Metodologías:

- 1: random_num con rand() y módulo, validando los argumentos. Para comprobarla, histograma de 100 000 números entre 1 y 10.
- 2: Fisher-Yates traducido del pseudocódigo adaptando los índices (posiciones de 0 a N-1, valores de 1 a N).
- 3: generate_permutations llama n_perms veces a generate_perm; si falla una, libera lo ya reservado.

- 4: InsertSort con un contador que suma una unidad cada vez que se evalúa la comparación de la OB.
- 5: generamos las permutaciones antes de medir, medimos con clock() y dividimos entre n_perms, de forma que ni la reserva de memoria ni la escritura del fichero entran en el tiempo.
- 6: BubbleSort on un contador que suma una unidad cada vez que se evalúa la comparación de la OB.

Pruebas:

- random_num con inf > sup, valores negativos y con inf = sup (devuelve ERR en los dos primeros casos y el propio valor en el tercero).
- Permutaciones: comprobamos que cada fila contiene los números de 1 a N sin repetir.
- InsertSort y BubbleSort: tabla de un solo elemento, tabla pequeña (-size 10), segmentos que no empiezan en 0 (comprobando que no se tocan los elementos de fuera) y argumentos inválidos (array NULL, ip negativo, iu menor que ip).
- generate_sorting_times con argumentos inválidos (incr = 0, num_min mayor que num_max, n_perms = 0
- Pruebas rápidas con tamaños pequeños antes de lanzar la tirada grande para además calentar el ordenador..

## 3. Apartados

Apartado 1:

Random_num devuelve un entero entre inf y sup, ambos incluidos, con rand() y el operador módulo. Devuelve ERR si inf > sup, si algún límite es negativo, si el intervalo es más grande de lo que cubre rand() (sup - inf > RAND_MAX) o si sup - inf vale INT_MAX, para no desbordar al calcular sup - inf + 1.

```c
int random_num(int inf, int sup)
{

  if(inf>sup || inf < 0 || sup < 0){
    return ERR;
  }

  if (sup - inf > RAND_MAX || sup - inf == INT_MAX){
    return ERR;
  }
  int diferencia = sup - inf + 1;
  return inf + rand() % diferencia;
}
```

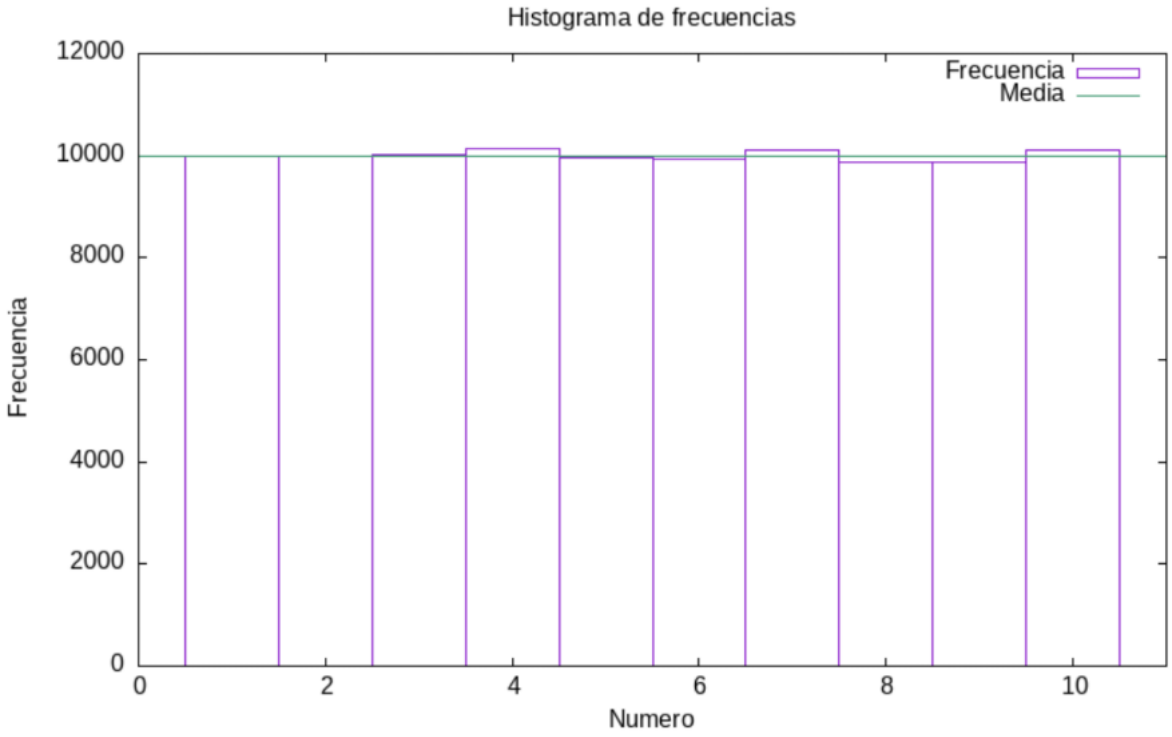
Para probarla elaboramos el histograma con GNU plot, generamos 100000 números para el intervalo [1, 10] (ambos incluidos), y obtenemos la cantidad de veces que hemos obtenido cada número:

| NÚMERO | NÚMERO DE VECES GENERADO |
|---|---|
| 1 | 9987 |
| 2 | 10002 |
| 3 | 10015 |
| 4 | 10126 |
| 5 | 9965 |
| 6 | 9939 |
| 7 | 10121 |
| 8 | 9881 |
| 9 | 9863 |
| 10 | 10101 |

Generamos el histograma gracias a un script de la inteligencia artificial Claude.AI, le pedimos que añadiera una recta con la frecuencia esperada para una distribución uniforme, al tener 10 números diferentes y haber generado un total de 100.000 números cada uno debería de haber salido 10.000 veces para una distribución uniforme (100.000/10 =10.000), le especificamos que en el eje x queríamos la frecuencia y que en el eje y queríamos los números, además del nombre del

histograma el nombre del archivo.txt con los datos y el formato del mismo, generándonos este script el cual esta explicado en la parte final de IA:

```
set term png size 800,500
set output "histograma.png"
set title "Histograma de frecuencias"
set xlabel "Numero"
set ylabel "Frecuencia"
set yrange [0:12000]
set xrange [0:11]
plot "Datos.txt" title "Frecuencia" with boxes, 10000 title "Media"
```




Como podemos ver tanto en los resultados en la tabla como en el histograma, no existe equiprobabilidad, pero vemos que para datos muy grandes está muy cerca de ella.

Apartado 2:

Generate_perm implementa Fisher-Yates. Los índices del array van de 0 a N-1 y los valores de la permutación de 1 a N, así que rellenamos perm[i] = i + 1 y en cada iteración intercambiamos perm[i] con perm[random_num(i, N-1)]. Devuelve NULL si N <= 0, si falla el malloc o si random_num devuelve ERR (liberando antes la memoria). La función swap_p es auxiliar.

```c
void swap_p(int *a, int *b)
{
  int aux = *a;
  *a = *b;
  *b = aux;
  return;
}

int *generate_perm(int N)
{
  if (N <= 0)
  {
    return NULL;
  }

  int *perm = (int *)malloc(sizeof(perm[0]) * N);
  if (perm==NULL)
    return NULL;

  for (int i = 0; i < N; i++)
  {
    perm[i] = i + 1;
  }

  for (int i = 0; i < N; i++)
  {
    int rnd = random_num(i, N-1);
    if(rnd==ERR){
      free(perm);
      return NULL;
    }
    swap_p(&perm[i], &perm[rnd]);
    // int aux = perm[i];
    // perm[i] = perm[rnd];
    // perm[rnd] = aux;
  }
  return perm;
}
```

Apartado 3:

Generate_permutations reserva un array de n_perms punteros y rellena cada posición con generate_perm, de forma que las permutaciones son independientes y se pueden repetir. Si algo falla a mitad, free_permutations libera las i permutaciones ya creadas y el array, y se devuelve NULL. Quien llama es responsable de liberar con free_permutations.

```c
int **generate_permutations(int n_perms, int N)
{
  if (N <= 0 || n_perms <= 0)
    return NULL;

  int **result = (int **)malloc(sizeof(result[0]) * n_perms);
  if (result==NULL)
  {
    return NULL;
  }

  for (int i = 0; i < n_perms; ++i)
  {
    result[i] = generate_perm(N);
    if (result[i]==NULL)
    {
      free_permutations(result, i);
      return NULL;
    }
  }

  return result;
}

void free_permutations(int **permutation, int n_permutations){
  if (permutation == NULL){
    return;
  }
  for (int i = 0; i < n_permutations; i++)
      {
        free(permutation[i]);
      }
      free(permutation);
      return;
}
```

Apartado 4:

InsertSort ordena de menor a mayor el segmento entre ip e iu, ambos incluidos y devuelve el número de veces que se ejecuta la OB (o ERR si los argumentos no son válidos).

La comparación array[j] <= aux es la OB. El contador se incrementa antes de evaluarla.

```c
int InsertSort(int *array, int ip, int iu)
{
  int OB = 0;
  if (array==NULL || ip < 0 || iu < ip)
    return ERR;
  for (int i = ip + 1; i < iu + 1; i++)
  {
    int aux = array[i];
    int j = i - 1;
    while (j >= ip)
    {
      OB++;
      if (array[j] <= aux)
      {
        break;
      }
      array[j + 1] = array[j];
      j--;
    }
    array[j + 1] = aux;
  }
  return OB;
}
```

Apartado 5

average_sorting_time genera n_perms permutaciones de tamaño N, las ordena con el método que le pasan y rellena la estructura TIME_AA. El tiempo es de CPU (clock) y es el de todas juntas (se hace fuera del for por que así aunque sumemos algo de tiempo el error es menor que el que se pueda dar si se hace de uno en uno ya que si cada llamada no llega a la unidad de tiempo mínima perdemos precisión) entre n_perms. La suma de OB se guarda en un long long para evitar desbordamientos. Si cualquier paso falla se libera la memoria y se devuelve ERR.

```c
short average_sorting_time(pfunc_sort metodo,
                           int n_perms,
                           int N,
                           PTIME_AA ptime)
{

  if (metodo == NULL || ptime == NULL || n_perms <= 0 || N <= 0)
  {
    return ERR;
  }

  long long OB_counter = 0;
  int OB_min = INT_MAX;
  int OB_max = 0;
  int aux;
  int **perms = generate_permutations(n_perms, N);

  if (perms == NULL)
  {
    return ERR;
  }

  clock_t ttime_ini = clock();
  if (ttime_ini == (clock_t)-1)
  {
    free_permutations(perms, n_perms);
    return ERR;
  }

  for (int i = 0; i < n_perms; i++)
  {
    aux = metodo(perms[i], 0, N - 1);
    if (aux == ERR)
    {
      free_permutations(perms, n_perms);
      return ERR;
    }
    if (aux > OB_max)
    {
      OB_max = aux;
    }
    if (aux < OB_min)
    {
      OB_min = aux;
    }
    OB_counter += aux;
  }
  clock_t ttime_fin = clock();
  if (ttime_fin == (clock_t)-1)
  {
    free_permutations(perms, n_perms);
    return ERR;
  }

  for(int i=0 ; i < n_perms; i++){
    for(int j=0; j < N-1; j++){
      if(perms[i][j] > perms[i][j+1]){
        free_permutations(perms, n_perms);
        return ERR;
      }
    }
  }

  ptime->N = N;
  ptime->n_elems = n_perms;
  ptime->time = (double)(ttime_fin - ttime_ini) / CLOCKS_PER_SEC / n_perms;
  ptime->average_ob = (double)OB_counter / n_perms;
  ptime->max_ob = OB_max;
  ptime->min_ob = OB_min;

  free_permutations(perms, n_perms);

  return OK;
}
```

generate_sorting_times recorre los tamaños N = num_min + k·incr mientras N <= num_max, llama a average_sorting_time para cada uno y guarda todo con save_time_table. Valida los argumentos, libera la tabla también si hay error y devuelve el resultado de save_time_table. save_time_table escribe una fila por tamaño con las seis columnas y comprueba los errores.

```c
short generate_sorting_times(pfunc_sort method, char *file,
                             int num_min, int num_max,
                             int incr, int n_perms)
{
  if(method == NULL || file == NULL || num_min < 1 || num_min > num_max || incr <= 0 || n_perms <= 0){
    return ERR;
  }
  // buscamos num de elementos tal que num_min + Kmax * incr <= num_max
  int kmax = (num_max - num_min) / incr + 1;
  PTIME_AA ptime = (PTIME_AA)malloc(sizeof(TIME_AA) * kmax);
  if (ptime == NULL)
  {
    return ERR;
  }
  int k = 0;
  while (1)
  {
    int N = num_min + k * incr;
    if (N > num_max)
      break;
    short result = average_sorting_time(method, n_perms, N, &ptime[k]);
    if (result == ERR){
      free(ptime);
      return ERR;
    }
    k++;
  }
  int ret_fun = save_time_table(file, ptime, k);
  free(ptime);
  return ret_fun;
}
```

```c
short save_time_table(char *file, PTIME_AA ptime, int n_times)
{
  if (file == NULL || ptime == NULL || n_times <= 0)
  {
    return ERR;
  }

  FILE *archivo = fopen(file, "w");

  if (archivo == NULL)
  {
    return ERR;
  }
  int counter=0;
  for (int i = 0; i < n_times; i++)
  {
    counter = fprintf(archivo, "%d %d %.9f %.6f %d %d\n", ptime[i].N, ptime[i].n_elems, ptime[i].time, ptime[i].average_ob, ptime[i].min_ob, ptime[i].max_ob);
    if(counter<0){
      fclose(archivo);
      return ERR;
    }
  }

  fclose(archivo);
  return OK;
}
```

Usamos un mínimo de 1000 elementos, un tamaño máximo de 10000 elementos, un incremento de 1000 elementos y 1000 permutaciones para cada tamaño. Así obtenemos los siguientes resultados para InsertSort:

| N | Tiempo medio (s) | OB media | OB mín | OB máx |
|---|---|---|---|---|
| 1000 | 0.000553866 | 250967.995 | 236426 | 268648 |
| 2000 | 0.002168737 | 1001020.617 | 955473 | 1046079 |
| 3000 | 0.004900013 | 2250302.607 | 2163662 | 2330982 |
| 4000 | 0.008620408 | 4002662.064 | 3884703 | 4145265 |
| 5000 | 0.013509975 | 6252545.931 | 6036227 | 6465790 |
| 6000 | 0.019497639 | 9006637.138 | 8774511 | 9243243 |
| 7000 | 0.026359245 | 12257922.701 | 11898203 | 12547364 |
| 8000 | 0.034498261 | 16006891.710 | 15575585 | 16371768 |
| 9000 | 0.043627217 | 20257548.543 | 19802496 | 20781259 |
| 10000 | 0.053910946 | 25012597.565 | 24431302 | 25589959 |

Como se puede observar sale una OB muy parecida a $\frac{n^2}{4}$ visto en teoría, por ejemplo para N=1000 $1000^2/4$ = 250000 que son aproximadamente los 250967,995 que hemos obtenido o por ejemplo para N=10000 que teóricamente son 25000000 de OB frente a los25012597,565 que hemos obtenido.

Apartado 6:

En BubbleSort la OB es la comparación array[j] > array[j + 1], que se ejecuta siempre antes de decidir si se intercambia.

```c
int BubbleSort(int *array, int ip, int iu)
{
  if (array == NULL || ip < 0 || iu < ip)
  {
    return ERR;
  }

  int OB = 0;
  for (int i = iu; i > ip; i--)
  {
    for (int j = ip; j < i; j++)
    {
      OB++;
      if (array[j] > array[j + 1])
      {
        swap_s(&array[j], &array[j + 1]);
      }
    }
  }
  return OB;
}
```

Al igual que con el exercise 4 para crearla duplicamos el exercise5.c lo renombramos llamandole exercise5b.c y cambiamos la llamada a BubbleSort, tras ejecutarlo obtenemos los siguientes resultados usando como mínimo 1000 elementos, un tamaño máximo de 10000 elementos, un incremento de 1000 elementos y 1000 permutaciones para cada tamaño al igual que con InsertSort para que los resultados sean comparables. Obtenemos así la siguiente tabla de resultados:

| N | Tiempo medio (s) | OB media | OB mín | OB máx |
|---|---|---|---|---|
| 1000 | 0.001746051 | 499500 | 499500 | 499500 |
| 2000 | 0.007050182 | 1999000 | 1999000 | 1999000 |
| 3000 | 0.016651525 | 4498500 | 4498500 | 4498500 |
| 4000 | 0.031251063 | 7998000 | 7998000 | 7998000 |
| 5000 | 0.051436739 | 12497500 | 12497500 | 12497500 |
| 6000 | 0.076985046 | 17997000 | 17997000 | 17997000 |
| 7000 | 0.107306841 | 24496500 | 24496500 | 24496500 |
| 8000 | 0.143496094 | 31996000 | 31996000 | 31996000 |
| 9000 | 0.184530779 | 40495500 | 40495500 | 40495500 |
| 10000 | 0.230776149 | 49995000 | 49995000 | 49995000 |

El número de OB es siempre $\frac{n(n-1)}{2}$ sin importar la permutación: para N = 1000 son 499 500 y para N = 10000 son 49 995 000, justo lo que sale. Por eso el mínimo, la media y el máximo coinciden en todas las filas. En este caso el mejor caso, el peor y el medio son iguales y valen $\Theta(n^2)$.

Si comparamos entonces para el caso más exacto de entre los casos de prueba (N = 10000) los dos algoritmos obtenemos:

| | InsertSort | BubbleSort |
|---|---|---|
| OB media | 25 012 598 | 49 995 000 |
| Tiempo medio (s) | 0.0539 | 0.2308 |

Podemos ver que BubbleSort hace casi 2 veces el número de OB que InserSort y que tarda aproximadamente 4 veces más. Por lo que vemos que sobre el papel es más efectivo usar InsertSort en cuanto a tiempo y número de OB realizadas.

# 4. Cuestiones

Cuestión 1

Le consultamos a Claude.AI si la fórmula inf + rand() % (sup - inf + 1) garantiza la equiprobabilidad y qué problema presenta bajo las hipótesis del apartado 1.

Identificación del problema (Sesgo del operador módulo / Modulo Bias): La implementación basada en el operador módulo no garantiza una distribución perfectamente uniforme cuando el número total de posibles valores generados por rand(), que es RAND_MAX + 1, no es un múltiplo exacto del tamaño del intervalo M = sup - inf + 1. Al aplicar la operación módulo rand() % M, los primeros (RAND_MAX + 1) % M valores del intervalo tienen una probabilidad ligeramente mayor de salir, ya que se les asigna un valor más del espacio de origen de rand().

Verificación con un ejemplo reducido: Supongamos un generador ideal reducido con RAND_MAX = 5 (genera equiprobablemente los 6 enteros {0, 1, 2, 3, 4, 5}) y queremos generar números en el intervalo [0, 3], cuyo tamaño es M = 4:

- Los valores 0 y 4 de rand() producen 0 (0 % 4 = 0 y 4 % 4 = 0). Probabilidad: 2/6 = 33,33%.
- Los valores 1 y 5 de rand() producen 1 (1 % 4 = 1 y 5 % 4 = 1). Probabilidad: 2/6 = 33,33%.
- El valor 2 produce 2 (2 % 4 = 2). Probabilidad: 1/6 = 16,67%.
- El valor 3 produce 3 (3 % 4 = 3). Probabilidad: 1/6 = 16,67%.

Los números 0 y 1 tienen el doble de probabilidad de aparecer que 2 y 3. En nuestro caso real, dado que RAND_MAX suele valer 2.147.483.647 y el rango probado es 10, el sesgo es prácticamente inapreciable en un histograma muestral, pero existe teóricamente.

Alternativa propuesta sin coma flotante (Muestreo por rechazo): Para eliminar completamente el sesgo sin recurrir a números reales ni perder precisión, se descartan

mediante rechazo aquellos valores de rand() que caen en el "resto" incompleto al final del rango [0, RAND_MAX]:

C

```
int random_num_sin_sesgo(int inf, int sup) {
   if (inf > sup || inf < 0 || sup < 0) return ERR;
   if (sup - inf > RAND_MAX || sup - inf == INT_MAX) return ERR;

   unsigned int M = (unsigned int)(sup - inf + 1);
   unsigned int limite = ((unsigned int)RAND_MAX + 1) - (((unsigned int)RAND_MAX +
1) % M);
   unsigned int r;

   do {
      r = (unsigned int)rand();
   } while (r >= limite);

   return inf + (int)(r % M);
}
```

Justificación y coste: Todos los valores aceptados pertenecen a un rango que es múltiplo exacto de M, por lo que cada residuo posible tiene exactamente el mismo número de preimágenes en rand(), garantizando equiprobabilidad estricta. En cuanto al coste, dado que RAND_MAX es inmensamente mayor que M, la probabilidad de rechazo es despreciable (menor que M / RAND_MAX) y el número esperado de iteraciones del bucle es prácticamente 1, por lo que el coste temporal sigue siendo O(1).

Cuestión 2

Le solicitamos a Claude.AI una propuesta de demostración formal de que el algoritmo de Fisher-Yates genera permutaciones equiprobables adaptada a nuestra indexación en C (random_num(i, N - 1)).

Demostración de equiprobabilidad: El algoritmo comienza con una permutación inicial fija de N elementos distintos [1..N]. Para que la distribución sobre el conjunto de todas las N! permutaciones posibles sea uniforme, cualquier permutación concreta debe obtenerse con probabilidad exactamente igual a 1 / N!.

El algoritmo realiza N pasos (índices i = 0, 1, ..., N - 1):

1. En la iteración i = 0, la llamada random_num(0, N - 1) elige una posición entre N candidatas con probabilidad 1 / N. El elemento elegido se intercambia con la posición 0, quedando fijado el primer elemento de la permutación.
2. En la iteración i = 1, la llamada random_num(1, N - 1) elige una posición entre las N - 1 restantes de manera uniforme e independiente, con probabilidad 1 / (N - 1).
3. De forma general, en el paso i, se elige uniformemente un elemento de entre las N - i posiciones restantes con probabilidad 1 / (N - i).

Aplicando la regla de la multiplicación para sucesos independientes, la probabilidad de fijar cualquier secuencia ordenada particular de N elementos es: P(permutación) = (1 / N) · (1 / (N - 1)) · (1 / (N - 2)) · ... · (1 / 2) · (1 / 1) = 1 / N!

Dado que existen exactamente N! permutaciones posibles y cada una se genera con probabilidad 1 / N!, el algoritmo produce permutaciones estrictamente equiprobables.

Efecto del sesgo en random_num: Si random_num no es perfectamente uniforme (por ejemplo, por el sesgo del módulo analizado en la cuestión 1), las probabilidades de transición 1 / (N - i) dejan de ser idénticas para todas las elecciones. Como consecuencia, ciertas ramas del árbol de ejecución tendrán mayor probabilidad que otras y la probabilidad final de algunas permutaciones será mayor que 1 / N!, perdiéndose la garantía teórica de equiprobabilidad.

Cuestión 3

Le pedimos a Claude.AI una propuesta de demostración formal de la corrección del algoritmo InsertSort a partir de nuestro código en C. Verificamos cada paso mediante la técnica del invariante de bucle aplicada al bucle exterior for (int i = ip + 1; i < iu + 1; i++).

Invariante del bucle: "Al inicio de cada iteración i del bucle exterior, el subsegmento array[ip .. i - 1] contiene los mismos elementos que ocupaban originalmente dichas posiciones, pero ordenados de forma monótona creciente: array[ip] ≤ array[ip + 1] ≤ ... ≤ array[i - 1]".

- Inicialización: Antes de la primera iteración, i = ip + 1. El subsegmento a considerar es array[ip .. ip], que consta de un único elemento. Un array de un solo elemento está ordenado por definición trivial, por lo que el invariante se cumple al inicio.
- Mantenimiento: Supongamos que el invariante es cierto para un índice i, es decir, array[ip .. i - 1] está ordenado. En la iteración actual se guarda aux = array[i]. El bucle interior while (j >= ip) compara aux hacia la izquierda desplazando una posición a la derecha todos los elementos que sean estrictamente mayores que él (array[j + 1] = array[j]). El bucle finaliza en cuanto encuentra el primer elemento menor o igual (array[j] <= aux) o cuando se alcanza el límite izquierdo del segmento (j < ip). Tras la parada, se inserta aux en la posición j + 1. Como todos los elementos a la izquierda de j + 1 son menores o iguales a aux, y todos los situados a su derecha son mayores o iguales, el nuevo subsegmento array[ip .. i] queda completamente ordenado y contiene los mismos elementos iniciales, preservando el invariante para la siguiente iteración.
- Terminación: El bucle for incrementa i de uno en uno hasta que i = iu + 1. Al terminar, sustituyendo i = iu + 1 en el invariante, se deduce que el subsegmento array[ip .. iu] contiene los elementos originales ordenados de menor a mayor, demostrando que la función ordena correctamente el intervalo especificado.

Terminación del bucle interior: En cada iteración del bucle while, el índice j decrece estrictamente en 1 unidad (j--). Dado que j parte de un valor finito i - 1 y está acotado inferiormente por la condición j >= ip, el número de iteraciones está acotado por i - ip, garantizando que el bucle interior siempre termina en tiempo finito.

Cuestión 4

Le consultamos a Claude.AI sobre los límites del bucle exterior de nuestra implementación de InsertSort (for (int i = ip + 1; i < iu + 1; i++)) y le pedimos evaluar si es válida la afirmación de que este bucle no actúa sobre el último elemento de la tabla.

Evaluación de la afirmación: La afirmación es rotundamente falsa. El bucle exterior sí actúa sobre el último elemento del segmento.

Justificación técnica:

- En nuestra implementación, la condición de parada del bucle es i < iu + 1 con incrementos de i++. Dado que trabajamos con aritmética entera, la condición i < iu + 1 es matemáticamente equivalente a i <= iu.
- Por tanto, la última iteración del bucle se ejecuta de manera efectiva con i = iu, procesando el último elemento de la tabla aux = array[iu] y desplazándolo hasta su posición ordenada definitiva dentro de array[ip .. iu].
- La confusión suele derivar de implementaciones donde el array tiene tamaño n y se itera de 1 a n - 1, o de algoritmos como BubbleSort cuyo índice exterior recorre solo hasta n - 1, pero en InsertSort el elemento final debe insertarse obligatoriamente para completar la ordenación.

Traza sobre una tabla pequeña:

Consideremos la tabla array = [3, 1, 2] con ip = 0 e iu = 2 (último elemento es array[2] = 2):

1. Iteración i = 1 (i < 3): Se procesa aux = array[1] = 1. Se compara con array[0] = 3 (3 > 1), se desplaza el 3 a la derecha y se coloca el 1 en la posición 0. La tabla queda [1, 3, 2].
2. Iteración i = 2 (i < 3): Corresponde a i = iu = 2 (el último elemento). Se procesa aux = array[2] = 2. Se compara con array[1] = 3 (3 > 2), se desplaza el 3 a la derecha, se compara con array[0] = 1 (1 <= 2, rompe el bucle) y se coloca el 2 en la posición 1. La tabla queda [1, 2, 3].
3. Paso siguiente i = 3: Ahora i < 2 + 1 resulta falso (3 < 3 no se cumple) y el bucle termina con la tabla completamente ordenada.

Queda demostrado empírica y formalmente que el bucle exterior sí actúa sobre el último elemento iu.

Cuestión 5

La operacion básica de InsertSort es la comparación array[j] <= aux dentro del while. Se ejecuta en todas las vueltas del bucle interior, así que sigue el coste del algoritmo: $n-1$ en el mejor caso y $n(n-1)/2$ en el peor. Le pasamos el código de nuestro InsertSort a Claude.AI y le pedimos que indicara la OB (coincidiendo con la nuestra) y propusiera otras operaciones básicas, pasándole nuestro código.

Alternativas propuestas por la IA y si valen:

Número de desplazamientos (array[j+1] = array[j]): no nos vale. En el mejor caso (tabla ordenada) hace 0, y el algoritmo sigue recorriendo la tabla (n-1 comparaciones), así que no refleja el coste lineal del mejor caso.

Número de intercambios (swaps): no vale para nuestra implementación, porque no hace swaps. Desplaza elementos y coloca aux al final. Es un error típico al describir el InsertSort.

Iteraciones del for exterior: siempre son n-1 sea cual sea la entrada, así que no captura el coste.

Iteraciones del while: vale, pero es lo mismo que nuestra OB, porque cada iteración hace una comparación.

Cuestión 6:

InsertSort (OB: array[j] <= aux)

- Mejor caso (tabla ordenada): una comparación por elemento, B_IS(n) = $n - 1$ que es Θ(n).
- Peor caso (tabla al revés): W_IS(n) = 1 + 2 + ... + (n-1) = $\frac{n(n-1)}{2}$ que es Θ(n²).

BubbleSort (OB: array[j] > array[j+1])

- Hace siempre las mismas comparaciones, sea cual sea la entrada: W_BS(n) = B_BS(n) = $\frac{n(n-1)}{2}$ que es Θ(n²).

Le pedimos a una IA (Claude.AI) que calcule el coste medio de ambos algortimos pansándole nuestro código.

Coste medio

- A_BS(n) = $\frac{n(n-1)}{2}$ que es Θ(n²), porque no depende de la entrada.

- A_IS(n): el número de comparaciones es OB = (inversiones) + (n-1) - R donde R son los elementos que llegan al principio de la tabla (en esos casos no hay comparación extra). Con permutaciones equiprobables:

  E[inversiones] = $\frac{n(n-1)}{4}$ porque cada pareja está invertida con probabilidad 1/2.

  E[R] = $\Sigma 1/(i+1)$ para i=1...n-1 que es, H_n - 1, porque el elemento de la posición i es el menor de los i+1 primeros con probabilidad 1/(i+1).

  Por tanto A_IS(n) = $n(n-1)/4 + (n-1) - (H_n - 1) = n^2/4 + 3n/4 - H_n$ que es Θ(n²).

Comparamos con los datos para N = 10000, la fórmula da 25 007 490 y con nuestros datos tenemos 25 012 597. Esa pequeña diferencia es, porque la teoría es un valor esperado y lo nuestro es la media de 1000 permutaciones (contra más permutaciones más cerca estaríamos de ese valor teórico). En BubbleSort coincide exactamente con $\frac{n(n-1)}{2}$ (49 995 000). Los mínimos y máximos de InsertSort quedan lejos del mejor y peor caso teóricos, porque es muy difícil que salgan en las permutaciones generadas aleatoriamente la tabla ordenada u ordenada al revés, además de que no tomamos una única permutación si no que tomamos varias de las cuales sale una media luego.

Aunque los dos son $\Theta(n^2)$ de media, pero BubbleSort hace aproximadamente el doble de comparaciones ($n^2/2$ frente a $n^2/4$). InsertSort se adapta a la entrada y BubbleSort, no.

## Cuestión 7:

Le pedimos a la inteligencia artificial (Claude.AI) que nos genere un script para representar las gráficas tal como se nos pide en la cuestión 7, para ello le pasamos el nombre de los archivos.txt y nos genera el siguiente script, con el que generamos las 3 gráficas:

```
set terminal pngcairo size 900,600
set encoding utf8
set grid
set key top left

ins = "exercise5output.txt"
bub = "exercise5boutput.txt"

# Columnas: 1 N, 2 n_elems, 3 time, 4 average_ob, 5 min_ob, 6 max_ob

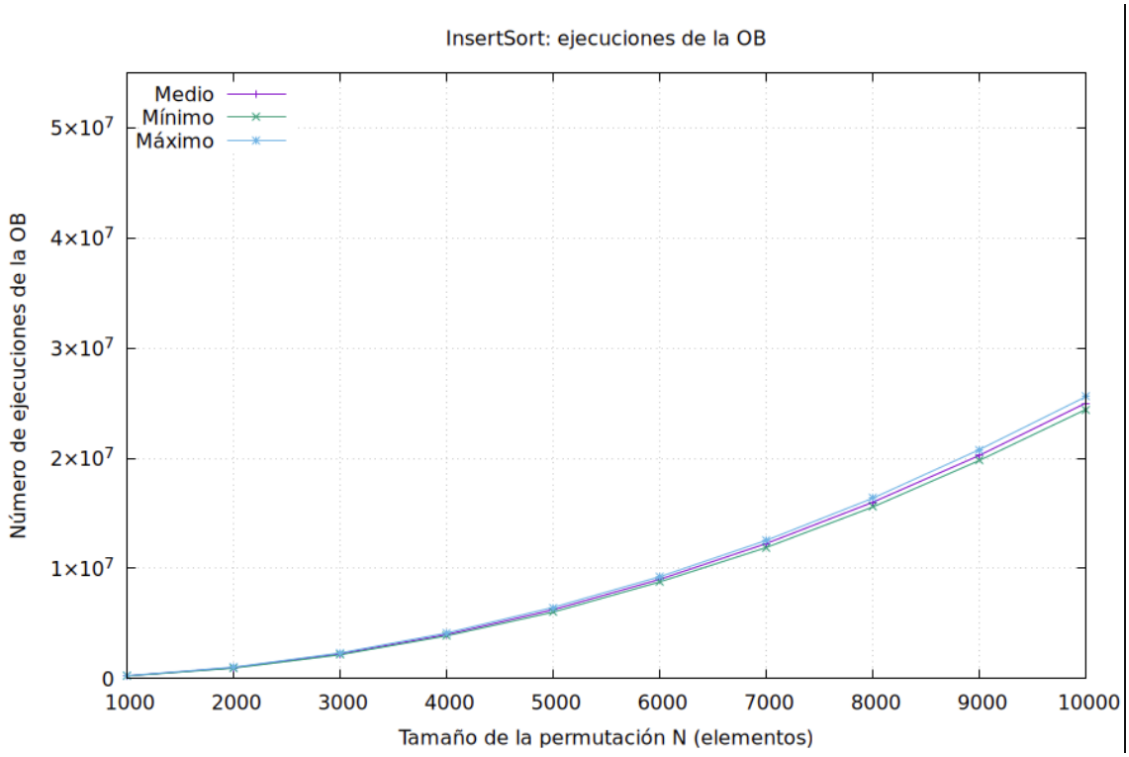
# 1 Tiempos medios de CPU de los dos algoritmos
set output "tiempos.png"
set title "Tiempo medio de CPU en función del tamaño"
set xlabel "Tamaño de la permutación N (elementos)"
set ylabel "Tiempo medio de CPU (s)"
plot ins using 1:3 with linespoints title "InsertSort", \
     bub using 1:3 with linespoints title "BubbleSort"

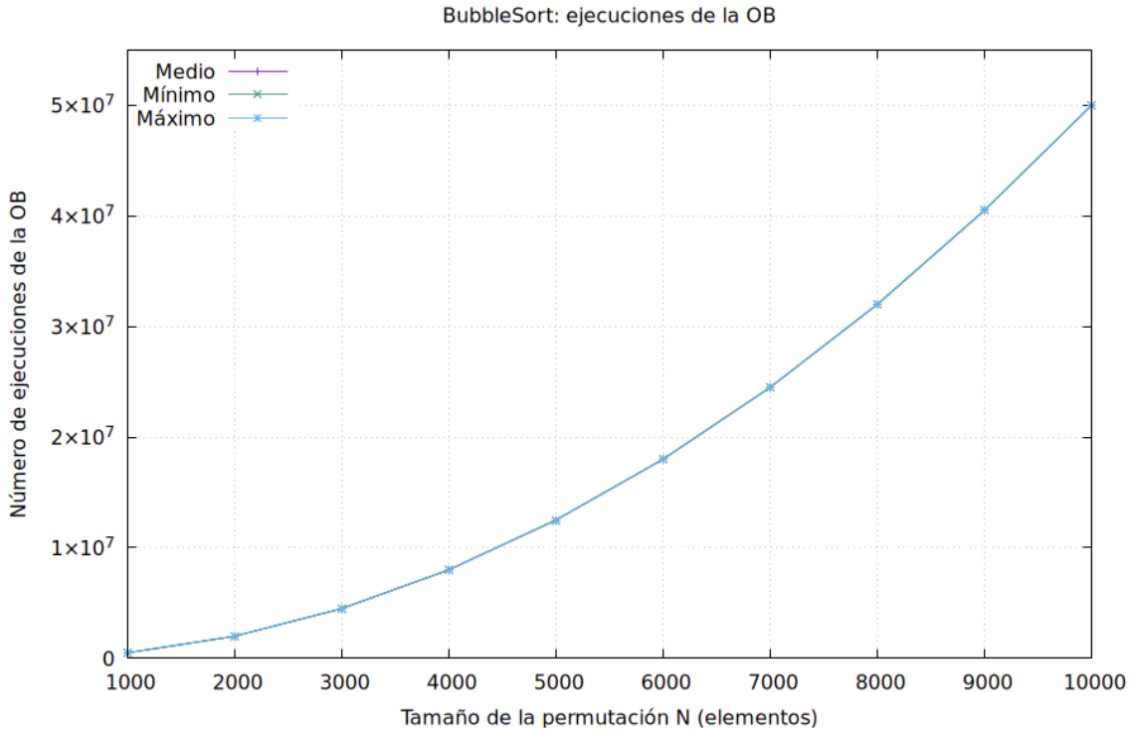
# Escala común para las gráficas de OB
set yrange [0:5.5e7]
set xlabel "Tamaño de la permutación N (elementos)"
set ylabel "Número de ejecuciones de la OB"

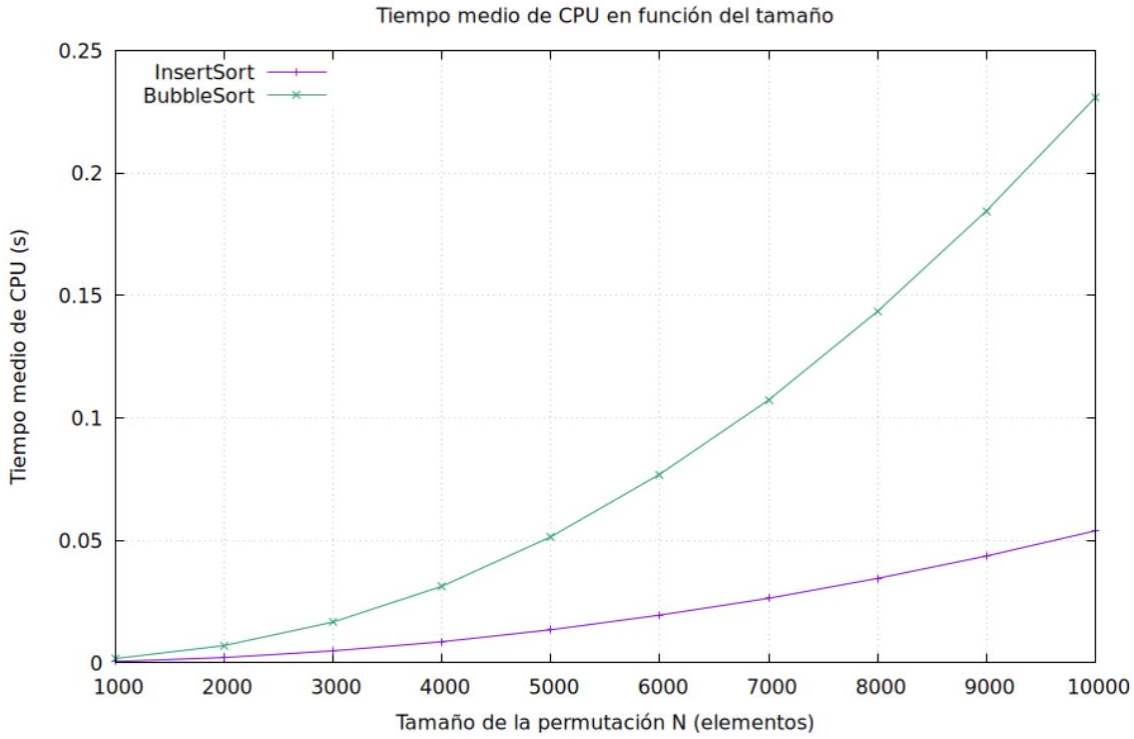
# 2 OB de InsertSort
set output "ob_insertsort.png"
set title "InsertSort: ejecuciones de la OB"
plot ins using 1:4 with linespoints title "Medio", \
     ins using 1:5 with linespoints title "Mínimo", \
     ins using 1:6 with linespoints title "Máximo"

# 3 OB de BubbleSort
set output "ob_bubblesort.png"
set title "BubbleSort: ejecuciones de la OB"
plot bub using 1:4 with linespoints title "Medio", \
     bub using 1:5 with linespoints title "Mínimo", \
     bub using 1:6 with linespoints title "Máximo"
```

A continuación, lo ejecutamos y analizamos las gráficas obtenidas:


InsertSort: ejecuciones de la OB
Medio
Mínimo
Máximo
Número de ejecuciones de la OB
0
1×10^7
2×10^7
3×10^7
4×10^7
5×10^7
1000
2000
3000
4000
5000
6000
7000
8000
9000
10000
Tamaño de la permutación N (elementos)


BubbleSort: ejecuciones de la OB
Medio
Mínimo
Máximo
Número de ejecuciones de la OB
0
1×10^7
2×10^7
3×10^7
4×10^7
5×10^7
1000
2000
3000
4000
5000
6000
7000
8000
9000
10000
Tamaño de la permutación N (elementos)




Como se puede observar en la última gráfica, las dos crecen de forma cuadrática y BubbleSort es más lento en todos los tamaños por ejemplo en N = 10000, 0,23 s frente a 0,054 s. Como ya habíamos dicho antes con las tablas de datos, BubbleSort hace el doble de comparaciones, pero tarda unas 4 veces más. Así que las comparaciones explican parte de la diferencia, y el resto probablemente venga de que BubbleSort hace además muchos intercambios.

Individualmente en la gráfica de InsertSort: la media sigue la curva $n^2/4$ de la teoría, y el mínimo y el máximo quedan casi pegados a ella (los casos extremos no aparecen en la muestra).

En cuanto a BubbleSort: las tres curvas coinciden, porque el número de comparaciones como ya hemos explicado en la cuestión anterior es siempre $\frac{n(n-1)}{2}$

Utilización responsable de la IA:

[Herramienta de IA utilizada; Consulta(s) realizada(s); Respuesta/propuesta relevante; Verificación realizada; Errores y limitaciones detectados y correcciones; Respuesta final razonada]

Además de las ya indicadas en la memoria:

Claude.AI:

Consulta 1:

Tengo un archivo.txt con una serie de datos (10 en total) y el número de veces que se repiten cada uno, necesito que me generes un script para general un histograma en

GNU PLOT,  el eje y debe ser la frecuencia, el eje x el número y además debe incluir la media que en este caso es 10000. Explícame además el código.

Repuesta 1:

./exercise1 -limInf 1 -limSup 10 -numN 100000 | tail -n +4 | sort -n | uniq -c | awk '{print $2, $1}' > Datos.txt

tail -> quita las cabeceras

sort -n | uniq -c -> cuenta cuantas veces sale cada numero en orden creciente

awk -> intercambia las columnas

gnuplot -e 'set term png size 800,500; set output "histograma.png"; set title "Histograma de frecuencias"; set xlabel "Numero"; set ylabel "Frecuencia"; set yrange [0:12000]; set xrange [0:11]; plot "archivo.txt" title "Frecuencia" with boxes, 10000 title "Media"'

set term png size 800,500 -> dimesiona la imagen salida

set output "histograma.png" -> le pone un nombre al archivo

set title "Histograma de frecuencias" -> Asigna un titulo a la grafica

set xlabel "Numero" -> titulo al eje x

set ylabel "Frecuencia" -> titulo al eje y

set yrange [0:12000] -> añade un dominio al eje y

set xrange [0:11] -> añade un dominio al eje x

plot "archivo.txt" title "Frecuencia" with boxes, 10000 title "Media" -> Lee el fichero datos, asigna el nombre frecuencia a la leyenda, dibuja la frecuencia como barras, dibuja una función constante con una línea de media.

Consulta 2:

Se le adjuntan a Claude.AI los ficheros.txt con los datos recogidos del ejercicio 5 tanto para InsertSort como para BubbleSort y se le nombra que es cada columna para que así genere las tablas con los datos para adjuntar así las capturas de estas en la memoria sin tener que ir metiendo cada resultado a mano en una tabla de Word.

## 5. Conclusiones finales

[Discusión final sobre la práctica y los resultados obtenidos.]

Tras el desarrollo e implementación de los algoritmos y la recopilación de mediciones empíricas, se extraen las siguientes conclusiones principales de la práctica:

1. **Generación uniforme y barajado de permutaciones:** El generador pseudoaleatorio random_num implementado mediante el operador módulo ofrece una aproximación suficientemente homogénea en intervalos pequeños para el análisis de algoritmos, como constató el histograma sobre 100.000 muestras con desviaciones inferiores al 1,4% respecto al valor medio teórico. Asimismo, el algoritmo de Fisher-Yates garantiza formalmente la generación equiprobable de permutaciones en tiempo lineal O(N) al desacoplar las elecciones progresivas sin sesgo estructural.
2. **Comportamiento y eficiencia de los algoritmos de ordenación:**
    a. Tanto InsertSort como la variante básica implementada de BubbleSort pertenecen a la clase de complejidad cuadrática en el caso medio y peor, $\Theta(n^2)$. Sin embargo, su comportamiento empírico y coste práctico divergen notablemente.
    b. Teóricamente, el número de comparaciones de la operación básica en el caso medio para InsertSort es aproximadamente $n^2 / 4$, mientras que en BubbleSort es siempre constante e insensible a la entrada con un valor exacto de $(n^2 - n) / 2 \approx n^2 / 2$. Los datos experimentales para N = 10.000 ratifican esta relación: InsertSort realizó una media de 25.012.597 OBs frente a las 49.995.000 OBs fijas de BubbleSort (el doble).
    c. En tiempo de ejecución de CPU, BubbleSort resulta casi cuatro veces más lento que InsertSort (0,23 s frente a 0,054 s para N = 10.000). Esta diferencia adicional de tiempo se debe a que BubbleSort ejecuta continuos intercambios en memoria (swap_s) con tres accesos a variables por cada elemento desordenado, mientras que InsertSort se limita a desplazar elementos mediante asignaciones directas (array[j + 1] = array[j]).
    d. Además, InsertSort es un algoritmo adaptativo cuyo mejor caso desciende a coste lineal $\Theta(n)$ para secuencias previamente ordenadas, propiedad de la que carece la implementación de burbuja sin parada anticipada.
3. **Metodología de medición y precisión:** La estrategia de medir tiempos acumulados por lotes de permutaciones mediante clock() y calcular posteriormente la media permite superar la limitación de la resolución temporal del reloj del sistema, eliminando del intervalo medido el impacto de la reserva dinámica de memoria y la escritura en disco, lo que arroja curvas empíricas que reflejan con gran fidelidad las predicciones asintóticas del análisis teórico.